



[Reference](#) › [Python Glossary](#) /

abstract base class (ABC)

In Python, an **abstract base class (ABC)** is a [class](#) that can't be [instantiated](#) on its own and is designed to be a blueprint for other classes, allowing you to define a common interface for a group of related classes.

ABCs allow you to define [methods](#) that must be created within any [subclasses](#) built from the abstract base class. This is useful for ensuring that derived classes implement particular methods from the base class, providing a consistent [interface](#) for different parts of your program.

To define an abstract base class, you inherit from `abc.ABC` and use the [@abstractmethod decorator](#) to mark methods that must be implemented by subclasses. Attempting to instantiate an abstract base class or a subclass that hasn't implemented all [abstract methods](#) will raise a [TypeError exception](#).

Examples

Here's an example of how to define and use an abstract base class in Python:

```
Python                                animals.py

from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass

    def jump(self):
        return f"{self.__class__.__name__} is jumping"
```



methods marked with `@abstractmethod`:

Python



```
>>> from animals import Animal

>>> class Dog(Animal):
...     pass

>>> Dog()
Traceback (most recent call last):
...
TypeError: Can't instantiate abstract class Dog
↳ without an implementation for abstract method 'speak'
```

Here's a correct implementation:

Python



```
>>> class Dog(Animal):
...     def speak(self):
...         return "Woof Woof"
...

>>> dog = Dog()
>>> dog.speak()
'Woof Woof'

>>> dog.jump()
'Dog is jumping'
```

`Dog` is a concrete class that inherits from `Animal` and implements the `.speak()` abstract method. It inherits the `.jump()` method from `Animal`.

Try it yourself. The interactive example below adds a second abstract method, `.move()`, to `Animal`, then lets you choose which abstract methods `Dog` implements. Watch how `Dog()` keeps raising a `TypeError` until every abstract method has an implementation:



```
animals.py

from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass

    @abstractmethod
    def move(self):
        pass

    def jump(self):
        return f"{self.__class__.__name__} is jumping"

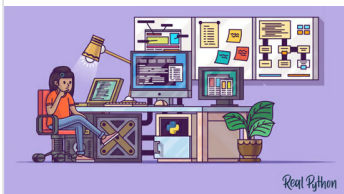
class Dog(Animal):
    pass
```

Python REPL

X Dog() raises TypeError

```
>>> from animals import Dog
>>> dog = Dog()
Traceback (most recent call last):
...
TypeError: Can't instantiate abstract class Dog without an
implementation for abstract methods 'move', 'speak'
```

Related Resources



Tutorial

Implementing Interfaces in Python: ABCs and Protocols

Learn how to implement interfaces in Python using abstract base classes, Protocols, and duck typing, and enforce method contracts cleanly.



For additional information on related topics, take a look at the following resources:

- [Python Classes: The Power of Object-Oriented Programming \(Tutorial\)](#)
- [Inheritance and Composition: A Python OOP Guide \(Tutorial\)](#)
- [Object-Oriented Programming \(OOP\) in Python \(Tutorial\)](#)
- [Python Interfaces: Object-Oriented Design Principles \(Course\)](#)
- [Implementing Interfaces in Python: ABCs and Protocols \(Quiz\)](#)
- [Class Concepts: Object-Oriented Programming in Python \(Course\)](#)
- [Inheritance and Internals: Object-Oriented Programming in Python \(Course\)](#)
- [Python Classes - The Power of Object-Oriented Programming \(Quiz\)](#)
- [Inheritance and Composition: A Python OOP Guide \(Course\)](#)
- [Inheritance and Composition: A Python OOP Guide \(Quiz\)](#)
- [A Conceptual Primer on OOP in Python \(Course\)](#)
- [Intro to Object-Oriented Programming \(OOP\) in Python \(Course\)](#)
- [Object-Oriented Programming \(OOP\) in Python \(Quiz\)](#)

By [Leodanis Pozo Ramos](#) • Updated July 9, 2026 • Reviewed by [Dan Bader](#)

[← Python Glossary](#)

[💬 Share Feedback](#)

Learn Python

[Start Here](#)

[Learning Resources](#)

[Code Mentor](#)

[Python Reference](#)

[Python Cheat Sheet](#)

Courses & Paths

[Learning Paths](#)

[Quizzes & Exercises](#)

[Browse Topics](#)

[Live Courses](#)

[Books](#)

Community

[Podcast](#)

[Newsletter](#)

[Community Chat](#)

[Office Hours](#)

[Learner Stories](#)



Membership

[Plans & Pricing](#)

[Team Plans](#)

[For Business](#)

[For Schools](#)

[Reviews](#)

Company

[About Us](#)

[Team](#)

[Mission & Values](#)

[Editorial Guidelines](#)

[Sponsorships](#)

[Careers](#)

[Press Kit](#)

[Merch](#)



[Privacy Policy](#) · [Terms of Use](#) · [Security](#) · [Contact](#)

❤️ Happy Pythoning!

© 2012–2026 DevCademy Media Inc. DBA Real Python. All rights reserved.
REALPYTHON™ is a trademark of DevCademy Media Inc.

